

-Kabir Sinha

-E23CSEU1343

CSET419 – Introduction to Generative AI

Lab 4: Character-Level Text Generation using LSTM and Transformer

```
[1]
✓ 9s      !pip -q install torch

import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import matplotlib.pyplot as plt

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(device)

▼      cuda
```

```
[2]
✓ 0s      text = """
artificial intelligence is transforming modern society.
it is used in healthcare finance education and transportation.
machine learning allows systems to improve automatically with experience.
data plays a critical role in training intelligent systems.
large datasets help models learn complex patterns.
deep learning uses multi layer neural networks.
neural networks are inspired by biological neurons.
each neuron processes input and produces an output.
training a neural network requires optimization techniques.
gradient descent minimizes the loss function.

natural language processing helps computers understand human language.
text generation is a key task in nlp.
language models predict the next word or character.
recurrent neural networks handle sequential data.
lstm and gru models address long term dependency problems.
however rnn based models are slow for long sequences.

transformer models changed the field of nlp.
they rely on self attention mechanisms.
attention allows the model to focus on relevant context.
transformers process data in parallel.
this makes training faster and more efficient.
modern language models are based on transformers.
"""

text = text.lower()
```

[3]

✓ 0s

```
chars = sorted(list(set(text)))
vocab_size = len(chars)

stoi = {ch:i for i,ch in enumerate(chars)}
itos = {i:ch for ch,i in stoi.items()}

def encode(s):
    return [stoi[c] for c in s]

def decode(l):
    return ''.join([itos[i] for i in l])

data = torch.tensor(encode(text), dtype=torch.long)
print("Vocab size:", vocab_size)
```

✓

Vocab size: 28

[4]

✓ 0s



```
seq_length = 40
X = []
Y = []

for i in range(len(data) - seq_length):
    X.append(data[i:i+seq_length])
    Y.append(data[i+1:i+seq_length+1])

X = torch.stack(X)
Y = torch.stack(Y)

print(X.shape, Y.shape)
```

✓

... torch.Size([1127, 40]) torch.Size([1127, 40])

[5]
✓ 0s

```
class LSTMModel(nn.Module):
    def __init__(self, vocab_size, hidden_size=128):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, hidden_size)
        self.lstm = nn.LSTM(hidden_size, hidden_size, batch_first=True)
        self.fc = nn.Linear(hidden_size, vocab_size)

    def forward(self, x):
        x = self.embed(x)
        out, _ = self.lstm(x)
        out = self.fc(out)
        return out

lstm_model = LSTMModel(vocab_size).to(device)
```

[6]
✓ 5s

```
optimizer = optim.Adam(lstm_model.parameters(), lr=0.003)
criterion = nn.CrossEntropyLoss()

epochs = 15
batch_size = 64

lstm_model.train()

for epoch in range(epochs):
    total_loss = 0

    for i in range(0, len(X), batch_size):
        xb = X[i:i+batch_size].to(device)
        yb = Y[i:i+batch_size].to(device)

        optimizer.zero_grad()
        output = lstm_model(xb)

        loss = criterion(output.view(-1, vocab_size), yb.view(-1))
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    print(f"Epoch {epoch+1}, Loss {total_loss:.4f}")
```



```
Epoch 1, Loss 52.9036
Epoch 2, Loss 43.4858
Epoch 3, Loss 38.2474
Epoch 4, Loss 34.1368
Epoch 5, Loss 30.3715
Epoch 6, Loss 27.0192
Epoch 7, Loss 24.3304
Epoch 8, Loss 22.0024
Epoch 9, Loss 19.7629
Epoch 10, Loss 17.5118
Epoch 11, Loss 15.2113
Epoch 12, Loss 13.3708
Epoch 13, Loss 11.8598
Epoch 14, Loss 10.3446
Epoch 15, Loss 9.3532
```

[7]
✓ 0s

```
def generate_text_lstm(model, start_text, length=200):
    model.eval()
    context = torch.tensor([encode(start_text)], dtype=torch.long).to(device)

    generated = start_text

    with torch.no_grad():
        for _ in range(length):
            output = model(context)
            probs = torch.softmax(output[0, -1], dim=0).cpu().numpy()
            idx = np.random.choice(len(probs), p=probs)

            generated += itos[idx]
            context = torch.cat([context, torch.tensor([idx]).to(device)], dim=1)
            context = context[:, -seq_length:]

    return generated

print(generate_text_lstm(lstm_model, "artificial intelligence "))
```



```
artificial intelligence is training fmod long comeratis used models are slow for lolows the insformers us madderres input and moders sodels are learning language
```

artificial intelligence is training fmod long comeratis used models are slow for lolows the insformers us madderres input and moders sodels are learning nequral networks are cused neural networks are oution pural on. Language

[8]
✓ 0s

```
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=500):
        super().__init__()
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2) * (-np.log(10000.0) / d_model))

        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)

        self.pe = pe.unsqueeze(0)

    def forward(self, x):
        return x + self.pe[:, :x.size(1)].to(x.device)
```

[9]
✓ 0s

```
class TransformerModel(nn.Module):
    def __init__(self, vocab_size, d_model=128, nhead=4, num_layers=2):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, d_model)
        self.pos = PositionalEncoding(d_model)

        encoder_layer = nn.TransformerEncoderLayer(d_model=d_model, nhead=nhead, batch_first=True)
        self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=num_layers)

        self.fc = nn.Linear(d_model, vocab_size)

    def forward(self, x):
        x = self.embed(x)
        x = self.pos(x)
        x = self.transformer(x)
        x = self.fc(x)
        return x

transformer_model = TransformerModel(vocab_size).to(device)
```

[10]
✓ 2s

```
optimizer_t = optim.Adam(transformer_model.parameters(), lr=0.003)

transformer_model.train()

for epoch in range(10):
    total_loss = 0

    for i in range(0, len(X), batch_size):
        xb = X[i:i+batch_size].to(device)
        yb = Y[i:i+batch_size].to(device)

        optimizer_t.zero_grad()
        output = transformer_model(xb)

        loss = criterion(output.view(-1, vocab_size), yb.view(-1))
        loss.backward()
        optimizer_t.step()

        total_loss += loss.item()

    print(f"Transformer Epoch {epoch+1}, Loss {total_loss:.4f}")
```

... Transformer Epoch 1, Loss 55.8110
 Transformer Epoch 2, Loss 49.0903
 Transformer Epoch 3, Loss 46.1332
 Transformer Epoch 4, Loss 44.8532
 Transformer Epoch 5, Loss 44.1229
 Transformer Epoch 6, Loss 43.5485
 Transformer Epoch 7, Loss 42.8549
 Transformer Epoch 8, Loss 42.2735
 Transformer Epoch 9, Loss 41.7825
 Transformer Epoch 10, Loss 41.4428

```
[11]
✓ 0s def generate_text_transformer(model, start_text, length=200):
      model.eval()
      context = torch.tensor([encode(start_text)], dtype=torch.long).to(device)

      generated = start_text

      with torch.no_grad():
          for _ in range(length):
              output = model(context)
              probs = torch.softmax(output[0, -1], dim=0).cpu().numpy()
              idx = np.random.choice(len(probs), p=probs)

              generated += itos[idx]
              context = torch.cat([context, torch.tensor([idx]).to(device)], dim=1)
              context = context[:-seq_length:]

      return generated

print(generate_text_transformer(transformer_model, "artificial intelligence "))
```

artificial intelligence dende mandren nntasreurntirdanthereunleueatepr
 leleucuet res ocs nt l mlare an nearess aguraneneralse pr chran

artificial intelligence dende mandren nntasreurntirdanthereunleueatepr
 moneraniandutaramoran ntkel mofolel utrk ng s ms sys lytodrkreenguelenely
 a wos utowows.
 leleucuet res ocs nt l mlare an nearess aguraneneralse pr chran

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import matplotlib.pyplot as plt
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(device)
```

```
text = ""
```

artificial intelligence is transforming modern society.

it is used in healthcare finance education and transportation.

machine learning allows systems to improve automatically with experience.

data plays a critical role in training intelligent systems.

large datasets help models learn complex patterns.

deep learning uses multi layer neural networks.

neural networks are inspired by biological neurons.

each neuron processes input and produces an output.

training a neural network requires optimization techniques.

gradient descent minimizes the loss function.

natural language processing helps computers understand human language.

text generation is a key task in nlp.

language models predict the next word or character.

recurrent neural networks handle sequential data.

lstm and gru models address long term dependency problems.

however rnn based models are slow for long sequences.

transformer models changed the field of nlp.

they rely on self attention mechanisms.

attention allows the model to focus on relevant context.

transformers process data in parallel.

this makes training faster and more efficient.

modern language models are based on transformers.

"""

```
text = text.lower()
```

```
chars = sorted(list(set(text)))
```

```
vocab_size = len(chars)
```

```
stoi = {ch: i for i, ch in enumerate(chars)}
```

```
itos = {i: ch for ch, i in stoi.items()}
```

```
def encode(s):
```

```
    return [stoi[c] for c in s]
```

```
def decode(l):
```

```
    return "".join([itos[i] for i in l])
```

```
data = torch.tensor(encode(text), dtype=torch.long)
```

```
print("Vocab size:", vocab_size)
```



```
seq_length = 40
```

```
X = []
```

```
Y = []
```

```
for i in range(len(data) - seq_length):
```

```
    X.append(data[i:i+seq_length])
```

```
    Y.append(data[i+1:i+seq_length+1])
```

```
X = torch.stack(X)
```

```
Y = torch.stack(Y)
```

```
print(X.shape, Y.shape)
```

```
class LSTMModel(nn.Module):
```

```
    def __init__(self, vocab_size, embed_size=64, hidden_size=128):
```

```
        super().__init__()
```

```
        self.embed = nn.Embedding(vocab_size, embed_size)
```

```
        self.lstm = nn.LSTM(embed_size, hidden_size, batch_first=True)
```

```
        self.fc = nn.Linear(hidden_size, vocab_size)
```

```
    def forward(self, x):
```

```
        x = self.embed(x)
```

```
        out, _ = self.lstm(x)
```

```
        out = self.fc(out)
```

```
        return out
```

```
lstm_model = LSTMModel(vocab_size).to(device)
```

```
criterion = nn.CrossEntropyLoss()
```

```
optimizer = optim.Adam(lstm_model.parameters(), lr=0.003)
```

```
batch_size = 64
```

```
epochs = 15
```

```
lstm_model.train()
```

```
for epoch in range(epochs):
```

```
    total_loss = 0
```

```
    for i in range(0, len(X), batch_size):
```

```
        xb = X[i:i+batch_size].to(device)
```

```
        yb = Y[i:i+batch_size].to(device)
```

```
        optimizer.zero_grad()
```

```
        output = lstm_model(xb)
```

```
        loss = criterion(output.view(-1, vocab_size), yb.view(-1))
```

```
        loss.backward()
```

```
        optimizer.step()
```

```
    total_loss += loss.item()
```

```

print(f"Epoch {epoch+1}, Loss {total_loss:.4f}")

def generate_text_lstm(model, start_text, length=200):
    model.eval()
    context = torch.tensor([encode(start_text)], dtype=torch.long).to(device)
    generated = start_text

    with torch.no_grad():
        for _ in range(length):
            output = model(context)
            probs = torch.softmax(output[0, -1], dim=0).cpu().numpy()
            idx = np.random.choice(len(probs), p=probs)

            generated += itos[idx]
            context = torch.cat([context, torch.tensor([[idx]]).to(device)], dim=1)
            context = context[:, -seq_length:]

    return generated

print(generate_text_lstm(lstm_model, "artificial intelligence "))

class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=500):
        super().__init__()
        pe = torch.zeros(max_len, d_model)

```

```
position = torch.arange(0, max_len).unsqueeze(1)
div_term = torch.exp(torch.arange(0, d_model, 2) * (-np.log(10000.0) / d_model))
```

```
pe[:, 0::2] = torch.sin(position * div_term)
pe[:, 1::2] = torch.cos(position * div_term)
```

```
self.pe = pe.unsqueeze(0)
```

```
def forward(self, x):
    return x + self.pe[:, :x.size(1)].to(x.device)
```

```
class TransformerModel(nn.Module):
```

```
    def __init__(self, vocab_size, d_model=64, nhead=4, num_layers=2):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, d_model)
        self.pos = PositionalEncoding(d_model)
        encoder_layer = nn.TransformerEncoderLayer(d_model=d_model, nhead=nhead)
        self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=num_layers)
        self.fc = nn.Linear(d_model, vocab_size)
```

```
    def forward(self, x):
        x = self.embed(x)
        x = self.pos(x)
        x = x.permute(1, 0, 2)
        out = self.transformer(x)
        out = out.permute(1, 0, 2)
```

```
out = self.fc(out)
```

```
return out
```

```
transformer_model = TransformerModel(vocab_size).to(device)
```

```
optimizer = optim.Adam(transformer_model.parameters(), lr=0.003)
```

```
epochs = 10
```

```
transformer_model.train()
```

```
for epoch in range(epochs):
```

```
    total_loss = 0
```

```
    for i in range(0, len(X), batch_size):
```

```
        xb = X[i:i+batch_size].to(device)
```

```
        yb = Y[i:i+batch_size].to(device)
```

```
        optimizer.zero_grad()
```

```
        output = transformer_model(xb)
```

```
        loss = criterion(output.view(-1, vocab_size), yb.view(-1))
```

```
        loss.backward()
```

```
        optimizer.step()
```

```
    total_loss += loss.item()
```

```

print(f"Transformer Epoch {epoch+1}, Loss {total_loss:.4f}")

def generate_text_transformer(model, start_text, length=200):
    model.eval()

    context = torch.tensor([encode(start_text)], dtype=torch.long).to(device)
    generated = start_text

    with torch.no_grad():
        for _ in range(length):
            output = model(context)

            probs = torch.softmax(output[0, -1], dim=0).cpu().numpy()
            idx = np.random.choice(len(probs), p=probs)

            generated += itos[idx]

            context = torch.cat([context, torch.tensor([[idx]]).to(device)], dim=1)
            context = context[:, -seq_length:]

    return generated

print(generate_text_transformer(transformer_model, "artificial intelligence "))

```